

第八次作业反馈

魏远毅

2024 年 11 月 13 日

1 问题 5.7

1.1 问题描述

已知一棵树如图 1 所示，请写出该树的先根序列和后根序列，并将该树转化为对应的二叉树。

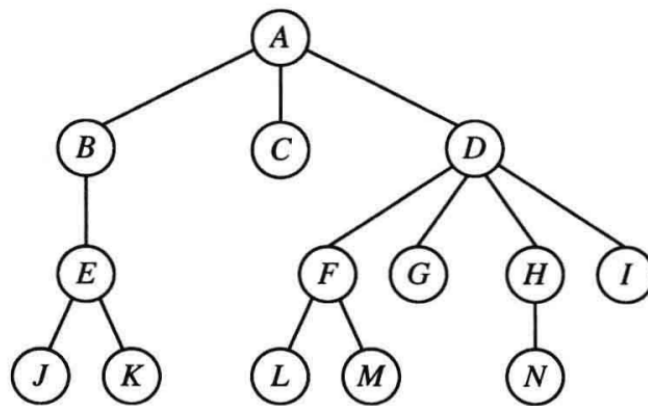


图 1: 问题 5.7

1.2 解答

先根序列遵循根结点-> 子树 1-> 子树 2->...-> 子树 n 的顺序 (子树编号遵循从左到右), 即先根序列为 ABEJKCDFLMGHNI。

后根序列遵循子树 1-> 子树 2->...-> 子树 n-> 根结点的顺序 (子树编号遵循从左到右), 即后根顺序为 JKEBCLMFGNHIDA。

将树转化为二叉树的方法:

- (1). 树中所有相邻的兄弟之间加一条线;
- (2). 对树中的每个非终端结点, 保留它与长子的连线, 删除它与其他孩子的连线;
- (3). 长子作为左孩子, 相邻的右兄弟作为右孩子。(或以树根为中心, 顺时针旋转 45 度)

补充: 部分同学没有注意到 (3), 即使二叉树的结点只有一个孩子, 这个孩子靠左和靠右也是不一样的, 比如结点 L 的孩子 M 就是一个右孩子。

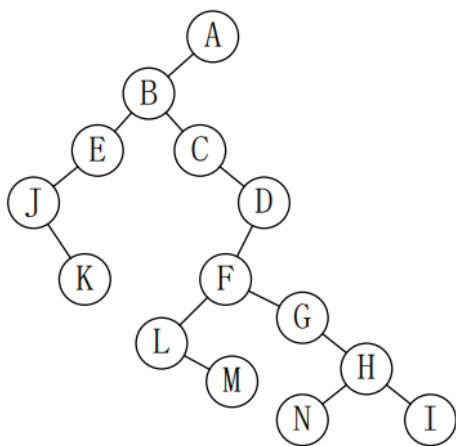


图 2: 问题 5.7: 树转化后的二叉树

2 问题 5.8

2.1 问题描述

将如图 2 所示的森林转化为对应的二叉树。

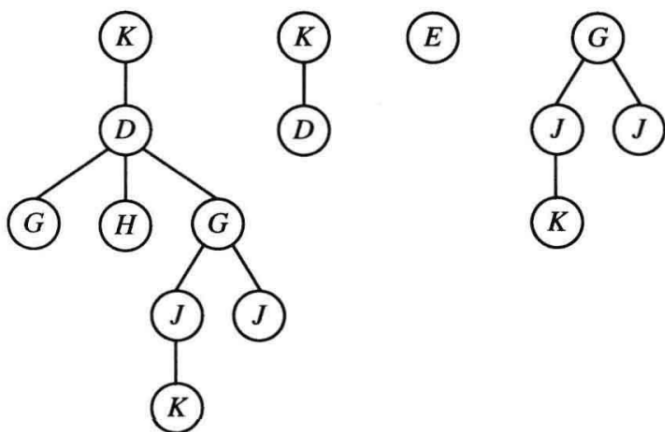


图 3: 问题 5.8

2.2 解答

森林其实和树差不多，我们可以构建一个虚拟的根结点 S，让它指向所有森林中的树的根结点，这样就得到了一棵树。

然后对这棵虚拟的树采用问题 5.7 中提及的方法，将它转化为二叉树。

此时会发现，二叉树的根结点为 S，且 S 只有左孩子 K，没有右孩子。

那么，我们只需要将 S 删除，让它的左孩子 K 变成新的根结点即可得到我们需要的二叉树。

补充：有几个同学询问过我重复的字母对于这个过程是否有影响，这可能是没分清 Huffman 树与二叉树的区别，Huffman 树是一种特殊的二叉树，对于常规的二叉树来说，结点的内容不会影响树的结构。

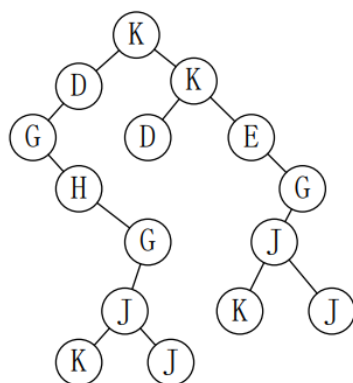


图 4: 问题 5.8: 森林转化后的二叉树

3 问题 5.13

3.1 问题描述

假设某个电文由 (a,b,c,d,e,f,g,h) 8 种字母组成, 每种字母在电文中出现的次数分别为 (7,19,2,6,32,3,21,10), 试解答下列问题:

- (1) 画出 Huffman 树;
- (2) 给出每个字母的 Huffman 编码;
- (3) 在对该电文进行最优二进制编码处理后, 给出电文的二进制位数。

3.2 解答

Huffman 树的构建方法:

- (1) 每个字母都可以视为一棵只有一个根节点的二叉树, 树的频率为该字母的频率, 这构成了一个二叉树集合 V ;
- (2) 从二叉树集合 V 中选出频率最低的两棵树 i 和 j , 将它们合并得到一颗新的二叉树 k (这里不关注左子树还是右子树), k 的频率为 i 和 j 的频率之和;
- (3) 将 (2) 中选出的 i 和 j 从二叉树集合 V 中剔除, 并将树 k 加入到集合 V 中;
- (4) 重复 (2) 和 (3) 的操作, 直到集合 V 中只剩下一棵树, 即为 Huffman 树。

根据上述方法, 可以绘制出 Huffman 树如图所示:

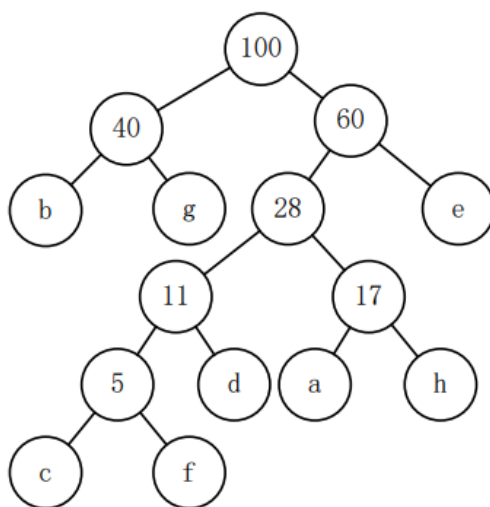


图 5: 问题 5.13: Huffman 树

根据已经构建好的 Huffman 树，可以得到每个字母的 Huffman 编码：(从根向下走，向左为 0，向右为 1)

a:1010 b:00 c:10000 d:1001 e:11 f:10001 g:01 h:1011

注意：因为我们在构建二叉树时不会强制要求左右子树的关系，所以构建的 Huffman 树可能不是唯一的，字母的编码也可能不同。

计算电文的二进制位数：($\sum$ 字母出现次数 * 字母编码长度)

$n=7*4+19*2+2*5+6*4+32*2+3*5+21*2+10*4=261(\text{bits})$

注意：尽管 Huffman 编码不是唯一的，但字母的编码长度一定是固定的，所有字母的编码的总长度一定是达到最小的。

4 问题 5.20

4.1 问题描述

写出统计树中叶子结点个数的算法，树用孩子兄弟链表表示。

4.2 解答

先回顾一下树的几种表示方法的节点结构：

(1) 双亲表示法：data+parent;

(2) 多重链表表示法：data+child1+child2+child3+...;

(3) 孩子链表表示法：孩子节点：data+next 表头结点：data+firstchild;

(4) 孩子双亲表示法：data+parent+firstchild;

(5) 孩子兄弟表示法：firstchild+data+nextsibling(右邻兄弟);

注意：部分同学可能把多重链表表示法和孩子链表表示法以及孩子兄弟表示法搞混淆了，还是需要注意一下几种表示方法的区别。

回到问题：

统计树的节点个数我们常常使用递归遍历的方法，而统计叶子节点的话其实就是在此基础上加一个叶子节点的判定条件。

Listing 1: 统计叶子节点数量

```
1 int num = 0; //全局变量记录叶子数量
2
3 void countLeaf(Tree *T){
4     if (!T) return; //空树
5     if (!T->firstchild) num++; //叶子节点
6     countLeaf(T->firstchild); //遍历第一个子树
7     countLeaf(T->nextsibling); //遍历下一个兄弟树
8     return;
9 }
```